計算物理学II

第4講：これまでの復習

秋山 進一郎

2024年10月25日

授業日の確認

- 第1講：10月4日（金）
- 第2講：10月11日（金）
- 第3講：10月18日（金）
- **第4講：10月25日（金）★**
- 第5講：11月1日（金）
- 第6講：11月15日（金）★
- 第7講：11月22日（金）★
- 第8講：11月25日（月）
- 第9講：12月6日（金）
- 第10講：12月13日（金）★

・★の付いた授業にてレポート課題を配布予定

今日の授業の目標

- これまでの授業の復習
  - レポートに取り組める環境は整っているか？
  - VS code, Overleafの利用
- GitとGitHubについて（オプション）

第1回レポート課題について

- lecture4の中に格納されています
  - report1.pdf（レポート課題）
  - report1ディレクトリ
    - report1.pdfのソースファイル
    - 課題で使う.bibファイル

LaTeX入門の簡単なドキュメント

- compphy2の中にintroduction_latex.pdfというドキュメントがあります
- 本日の講義に限らず，必要に応じて適宜参照してください
- ミニマルな情報のみなので，適宜ネット検索も

毎回の授業の流れ

“compphy2”に移動し，“git branch”で確認

↓

“git fetch”と“git merge”で演習資料を入手

↓

“lecture*”に移動し，
“lecture_material_*.pdf”を開いて演習開始

本日の演習内容

- これまでの復習
  - VS code, Overleafの利用
  - レポート課題の確認
- GitとGitHubについて（オプション）

ブランチを切る

- 原則, GitではmainブランチでI作業しない
  - “exercise”という名前のブランチを作成（①）
  - mainからexerciseへの切り替え（②）
  - 自分がいるブランチの確認（③）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch exercise   ①
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git switch exercise   ②
Switched to branch 'exercise'
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch            ③
* exercise
  main
```

LaTeXが使える環境は整っていますか？

- まだの人は，まずは第3回の演習資料に沿って設定を進めましょう
- Overleafも使えるようにしておくと楽でしょう
- ここまでできている人は，レポート課題の内容を確認しましょう
  - report1.pdfを開き，内容を確認する

レポート課題の進め方

- 第3回の講義で配布したサンプルを使ってレポートを書き始めるのが楽でしょう
  - lecture3の中のsample_latexというディレクトリをコピーする
    - コピー時にディレクトリ名を変更する場合，名前の衝突に注意
  - Overleafを使う場合，lecture3の中のtemplate_overleaf.zipをアップロードする
  - 「LaTeX入門」の1節も参照のこと
- 慣れないうちは難しく感じると思います
  - 積極的に周りの人と相談する，スタッフやTAを頼る

以降の演習

- この時間を使ってレポート課題に取り組んでもらってOKです
- 授業の最後に作業用ブランチを消すのを忘れずに
- 以降のスライドには，GitとGitHubに関する簡単な資料が載せてあります
  - 以降の内容は発展的な内容なので，興味のある人向け
  - 以降のスライドを読んでいなくても，今後の授業には全く問題ありません

作業が終わったら作業用ブランチを消す

- 実際のプログラム開発では, 作業用ブランチで新たに作成したファイル等もGitの管理下に置くが, 今日の講義ではやりません
  - mainブランチに戻る（①）
  - exerciseブランチを削除（②）
  - 自分がいるブランチの確認（③）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch -d exercise
Deleted branch exercise (was 2bf0644).
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch
* main
```

①

②

③

Gitとはバージョン管理システムの一つ

- バージョン管理システムとは？
  - ファイルのバージョン（更新履歴）を管理するためのツール
  - いつ，誰が，何を修正したのか，その履歴を管理してくれる
  - 必要となれば古いバージョンを参照できる
- 以下のような場面で役にたつ，あるいは不可欠なツール
  - 複数人でのドキュメント作成，プログラム開発
  - 一人でも複数の場所（パソコン）でドキュメント作成，プログラム開発をする場合
    - USBメモリでデータを持ち歩く必要がない
  - バグの発見と修正

バージョン管理システムを使わない場合

- ファイル名でバージョンを管理する
  - ファイルを修正するごとに名前を変えないといけない
  - いつ，誰が，何を修正したのか分からなくなる
  - どれが最新版なのか分からなくなる

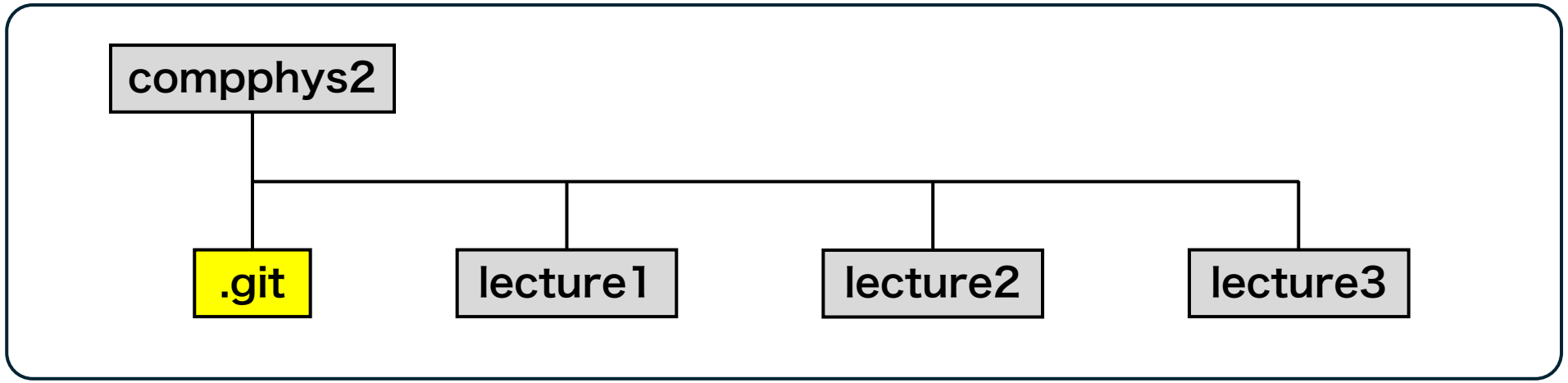
リポジトリ 1/2

・Gitでは，ファイルの更新履歴をリポジトリ単位で管理する

・リポジトリの実体は，.gitという名前のディレクトリ

例）授業で使っているcompphys2

・compphy2に入っている全てのファイルがGitの管理下にある



リポジトリ 2/2

- リポジトリには二種類ある
  - ローカルリポジトリ
    - 自分の手元の計算機，パソコンに存在する
  - リモートリポジトリ
    - リモートサーバ上に存在する
    - リモートリポジトリとしては，GitHubがよく用いられている
- Gitでは，ローカルリポジトリとリモートリポジトリを連携して使う

GitHubのアカウントを作る

- ブラウザからGitHubにアクセスし，Sign upする
  - アカウント名とパスワードの設定をする
- 手元のTerminalからGitHubへアクセスできるようにする
  - SSH公開鍵認証を設定する
    - ローカルリポジトリでの修正内容をリモートリポジトリ（GitHub）へ反映させる場合，その修正を送ってきた人物がリモートリポジトリにアクセスする権利を持っているかを確かめるため
    - 知らない人が勝手に自分のリモートリポジトリに修正を送ってくることを防ぐ

SSH公開鍵認証

- 公開鍵と秘密鍵のペアを作る
  - 公開鍵をGitHubに登録する
  - 秘密鍵はローカルに置いておく
  - 秘密鍵を持っている人だけが，対応する公開鍵の置かれた場所にアクセスできる
  - 秘密鍵は絶対に外部へ公開しないこと

公開鍵と秘密鍵の作成 1/2

- 以下のコマンドを実行する

```
akiyama.shinichir.ga@icho01:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/akiyama.shinichir.ga/.ssh/id_ed25519):
```

- 公開鍵と秘密鍵の保存場所を尋ねられているが，ここではこのままホームディレクトリの.sshというディレクトリに鍵のペアを作成する
  - 上の画面の状態でEnterを押す

公開鍵と秘密鍵の作成 2/2

- Enterを押すと次のようになる

```
Enter passphrase (empty for no passphrase):
```

- 秘密鍵にパスフレーズをつける
  - パスフレーズを入力したら，Enterを押す
  - 確認のためもう一度同じパスフレーズを入力し，Enterを押す

```
Enter same passphrase again:
```

- 次のようになったら公開鍵と秘密鍵の作成が完了

```
Your identification has been saved in /home/akiyama.shinichir.ga/.ssh/id_ed25519
Your public key has been saved in /home/akiyama.shinichir.ga/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:AQtd5ao0VknPp3iGWTDFTRbsllnBbfjccC6zm5vuzEU akiyama.shinichir.ga@icho01
The key's randomart image is:
+--[ED25519 256]--+
|    ....=+o++o.+ |
|     ..+ B..o = +|
|      . + =..+ B.|
|       . B o= o =|
|      + S +.   +E|
|     o o o    .. |
|      .        o.|
|             ooo |
|             oB. |
+----[SHA256]-----+
```

公開鍵をGitHubに登録する

- ブラウザからGitHubにアクセスし，以下の手順で公開鍵を登録する

① 右上のアイコンをクリックし，下の方にある「Settings」をクリック

② 左側の一覧から「SSH and GPG keys」をクリック

③ 右上緑色の「New SSH key」をクリック

④ TitleとKeyを入力する

- Titleはなんでもよい．Keyには公開鍵の内容を貼り付ける
- 公開鍵の内容を見るには，自身のTerminalでcat ~/.ssh/id_ed25519.pub
- 必ずid_ed25519.pubをコピーすること（ id_ed25519は秘密鍵である ）

⑤ TitleとKeyの入力が終わったら，緑色の「Add SSH key」をクリック

公開鍵登録の確認

- Terminalで以下のコマンドを実行する

```
akiyama.shinichir.ga@icho01:~/.ssh$ ssh -T git@github.com
```

- もし，以下のようなメッセージが出たらyesと入力してEnterを押す

```
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

- もし，Enter passphrase for key … と出たら公開鍵と秘密鍵の作成時に設定したパスフレーズを入力してEnterを押す

- 以下のメッセージが出ればOK（akiyama-esの部分には自分のアカウント名が出る）

```
Hi akiyama-es! You've successfully authenticated, but GitHub does not provide shell access.
```

リポジトリの作成

- ブラウザからGitHubのホーム画面に移動する
  （左上の猫のアイコンをクリック）
- 画面の左の方にある緑色の「New」をクリック
- 右のような画面になるので以下を設定

① リポジトリの名前（英数字のみ）

② リポジトリの説明（任意）

③ リポジトリの公開設定
とりあえず「Private」を選らぶ
「Public」だと公開されるので注意

④ リポジトリ作成時に作るものを選ぶ
とりあえず「Add a README file」に
チェックをするだけでいい

⑤ 「Create repository」をクリック

作成したリポジトリをローカルにクローンする 1/2

- 「Create repository」をクリックするとリポジトリの画面に移ったはず
  - 右上の緑色の「Code」をクリックする
  - 「Clone」のウィンドウが現れたら，「SSH」を選択
  - URLが現れるので，右にあるコピーボタンを押してコピーする

作成したリポジトリをローカルにクローンする 2/2

- 以下のように，ホームディレクトリにリポジトリをクローンしてみよう
  - 授業で使っているcompphys2ディレクトリの中ではgit cloneしないように

```
akiyama.shinichir.ga@icho01:~/.ssh$ cd
akiyama.shinichir.ga@icho01:~$ git clone git@github.com:akiyama-es/test.git
Cloning into 'test'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

ローカルでの修正をリモートに反映させる 1/2

- git cloneしたファイルに入ると，README.mdファイルがある
  - これを編集してみましょう
  - VS codeなどを使って好きな文言を入力し，保存する

ローカルでの修正をリモートに反映させる 2/2

・README.mdの編集内容をgit addし，git commitする

① git add README.md を実行

② git commit –m “コミットメッセージ” を実行

・コミットメッセージには修正内容の記録を残す

② git push を実行

```
akiyama.shinichir.ga@icho01:~/test$ git add README.md
akiyama.shinichir.ga@icho01:~/test$ git commit -m "README.md is updated"
[main bfb2519] README.md is updated
 1 file changed, 2 insertions(+), 1 deletion(-)
akiyama.shinichir.ga@icho01:~/test$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 275 bytes | 275.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To github.com:akiyama-es/test.git
   78b53be..bfb2519  main -> main
```

リモートリポジトリの確認

- ブラウザでGitHubのリポジトリをもう一度見てみよう
  - ブラウザをリロードする
  - README.mdの内容が更新されているはずである

複数の場所から同じリポジトリにアクセスする場合

- 先ほど同様に，ローカルで鍵のセットを作り，SSH公開鍵認証の登録をする
- リポジトリをローカルにクローンする（git clone）
  - git cloneは一度だけ
- クローンしたリポジトリ内で作業を進める
- 作業が終わったら，git add, git commit, git push

リモートレポジトリの更新履歴をローカルに反映する

- すでにローカルにクローンされたリポジトリがあり，リモートが更新されいる場合
  - リモートの更新内容をローカルに反映させる
  - git fetch, git merge をする
- その後，ローカルリポジトリ内で作業を進める
- 作業が終わったら，git add, git commit, git push